Job-Board Scraping Platform

Design White Paper — v3

Michael Tabet

2026-07-13

Abstract

A ground-up redesign of the job-board scraping platform (currently atlas-kt: Kotlin + Temporal, ∼13,269 employer boards across 13+ ATS platforms, growing). The new system is Python, built from five independent parts: a scraper library, ClickHouse storage, Temporal execution, an Airflow janitor/calibrator, and a Django control plane. Doctrine: *extract don't parse; fail loud, handle daily; no invented constants — every sizing is a hypothesis audited by data; observability means validation gates, not dashboards of counts.* The two systems run in parallel under a strict board-ownership rule until cutover, platform by platform.

Contents

1 The Problem

The current system works and is a black box to its owner. Measured facts, not impressions:

- 56 scraper classes implement a raw interface directly. **Zero shared base class.** The original Python system had a `BaseScraper`; the Kotlin port deliberately flattened it into every file.
- **37 scrapers carry a copy-pasted `stripHtml()`; 42 carry their own `sha256Hex()`.** Every scraper re-implements pagination, retries, and date handling. One behavioral fix touches up to 42 files.
- The owner cannot read Kotlin/JVM code. All inspection and repair routes through an intermediary — unacceptable for a system whose steady-state workload *is* repair (§3.1).
- **Silent failure history.** 2026-07-02: ∼500 boards reported success while publishing nothing (empty bodies validated cleanly; `jobs=[]` passes a schema check).
- **Retry lockup history.** 2026-07-03: unlimited retries on one permanently-403'd board froze an entire nightly run after ∼36 boards.

Decision: build a new system in Python; run both in parallel under strict board ownership (§12). This is not a port of code — it is a port of *knowledge*: each platform's quirks are documented facts in the old code's comments, paid for by incidents, and they transfer as facts.

2 Doctrine

Five commitments that settle later arguments:

1. **Readable by the owner.** Python; a hierarchy of at most three levels; every behavior has exactly one home file (§5.3).

2. **Extract, don't parse.** Scrapers land raw bytes. Parsing is a separate, replayable batch concern, deliberately deferred.
3. **Fail loud, handle daily.** No defensive tuning inside Temporal. If something breaks, it breaks visibly; a daily Airflow pass reads the evidence and deals with it. We would rather a red workflow and a notification than a silent constraint.
4. **No invented constants.** Any number (slots, batch sizes, budgets) is a *hypothesis*, recorded as such, audited and re-sized by data on a schedule (§8.2). Laws are not knobs; knobs are not laws.
5. **Observability = validation gates.** Not counters, not vanity dashboards: explicit assertions with defined failure actions (§10). If a signal does not change a decision, it does not exist.

3 The Five Hard Problems

3.1 Drift — pages change under us

ATS platforms change APIs, markup, and anti-bot posture continuously. Consequences: a platform change must touch exactly one file (blast radius); breakage must state *where and what* ("workday: detail fetch 403, 0/1400 jobs" — never "run complete ✓"); repair must be reproducible offline from recorded fixtures (§5.5).

3.2 Cost — boards are not equal

A Greenhouse board is 1 request; a 1,400-job Workday board is ∼1,407. Budgeting must be per-board and learned from history (§8.3).

3.3 Redundant compute — the dedup tradeoff

Blind re-scraping re-fetches thousands of unchanged job details nightly. Details are ∼99.5% of a heavy board's cost; ∼95% of stubs are unchanged day-over-day. Resolved by incremental extraction (§8.4) without compromising the raw-first architecture.

3.4 Growth — boards keep arriving

The employer list grows continuously. Every run reads the board table fresh at start; a board added in the Django admin is in that night's run. No deploys; no magic numbers anywhere in the architecture.

3.5 Orchestration overhead at scale

Per-board workflows at 13K+ boards ≈ 250–350K Temporal history events per night against Temporal's Postgres — on a cluster with a documented HDD IOPS starvation failure mode. Task granularity is a sized hypothesis (§7).

4 The System in Five Parts

Part	Responsibility	Depends on
1. Scraper library	Python classes; board → raw bytes + evidence	nothing
2. Storage	ClickHouse: raw payloads, evidence ledger, CDC metadata	nothing
3. Trigger	Temporal: schedules, fan-out, execution, recording	1, 2
4. Janitor/Calibrator	Airflow: re-runs, gates, sizing, parsing	2
5. Control plane	Django: boards, ownership, switches, budget intents	2

Parts 1 and 2 depend on nothing and are buildable today. Each part is designed, built, and finished without reference to the others beyond its declared interface.

5 Part 1 — The Scraper Library

5.1 The six acquisition families

The classification that matters is *how the data is acquired* — the family is where shared code lives:

#	Family	Shape	Examples
1	ONE-SHOT	one call returns everything	Greenhouse, Lever
2	PAGED	walk pages, each complete	Recruitee-style
3	PAGED+DETAIL	pages of stubs, then per-job detail	Workday, iCIMS
4	HTML	no API; parse served HTML	older ATSes
5	BROWSER	JS-built page; render headless	Paradox (the only one today)
6	BROWSER+SESSION	login / anti-bot	none yet — slot reserved

Ground truth from the current registry: **54 of 55 platforms are plain HTTP**. Browser scraping is a bolt-on, not the core. *Protection level* (Cloudflare, TLS fingerprint, headers) is deliberately not a family — it is client configuration on any family; folding it in would mint a dozen fake families.

5.2 The class hierarchy — three levels

```
L1  Scraper                              contract: (board, ctx) -> RawResult
L2  OneShotScraper / PagedScraper / PagedDetailScraper /
    HtmlScraper / RenderScraper / SessionScraper       <- THE LOOPS, once each
L3  WorkdayScraper(PagedDetailScraper) etc.            <- platform FACTS only
```

Transport (`HttpClient`, `BrowserClient`) is *composition*: injected via a context object, never inherited. Family loops call `ctx.http`; they never construct clients; tests inject a fixture-replaying fake.

Rules that keep the tree from rotting:

- **Family = mechanism, platform = facts.** An `if platform == "workday"` inside a family file is a design failure; that logic is a platform override.
- **Inheritance only for the template chain.** Utilities (date parse, HTML strip, digest) are pure functions in `normalize.py` — never mixins, never base-class methods.
- **The pagination hook is a cursor:** `parse_list(resp)` → `(stubs, cursor|None)`. Covers offset (Workday), page-number, token, and one-shot (always `None`) with a single shape. No platform overrides the loop itself.

5.3 Where-is-what: the locality contract

Readability is enforceable only if every question has a one-file answer. This table ships in the repo README:

"Where is…?"	Exactly one home
rate caps / headers / TLS / connection handling	`core/http.py`
the scraping loop for a family	`core/families/<family>.py`
a platform's URLs, cursor, quirks	`platforms/<platform>.py`
date/HTML/digest utilities	`core/normalize.py`
what a scraper returns	`core/models.py`
error taxonomy	`core/errors.py`
platform → class mapping	`platforms/registry.py`

If a behavior ever has two plausible homes, that is a design bug to fix, not a judgment call to live with.

5.4 Evidence-carrying results

`fetch()` returns payloads *plus* the run's evidence:

```
RawResult: payloads[], http_status, pages_fetched, stubs_seen,
           details_ok, details_failed, bytes_in, errors[]
```

Judgment happens *outside* the scraper, against this evidence (§10). Zero rows is distinguishable from success at the type level; the 2026-07-02 silent-success failure class becomes structurally impossible.

5.5 The test story — CI gates, not conventions

1. **No platform exists without fixtures.** A registry-coverage test walks `platforms/` and fails CI if any platform lacks: ≥1 recorded list response, ≥1 recorded detail response (family 3), and an expected `RawResult` snapshot.
2. **Recording is a one-liner:** `scrape record <platform> --board <id>`. Re-recording after drift is the *first* repair step — the fixture diff usually is the diagnosis.
3. **Family contract tests** exercise each loop once against synthetic fixtures (empty page, short last page, cursor loop, partial detail failures); platforms do not re-test the loop.
4. **Standalone runnability:** `scrape run <platform> --board <id> --dry` — no Temporal, no ClickHouse; evidence printed to stdout. This is the 2am tool.

6 Part 2 — Storage (ClickHouse)

6.1 Tables

- `scrape_raw` — append-only, immutable: `platform, board_id, url, kind(list|detail), http_status, fetched_at, run_id, payload, digest(sha256), stub_digest`. `stub_digest` exists from day one (§8.4 depends on it; painful to backfill).
- `scrape_evidence` — one row per board per run: the `RawResult` counts + outcome. This table is simultaneously the cost ledger, the gate input, and the observability source.
- `temporal_meta` — CDC-replicated Temporal workflow metadata (§6.3).
- `gate_results` — every gate evaluation (§10) is itself a row: gates are data.

Engine note: `ReplacingMergeTree` merges lazily — any dedup-dependent read (seen-sets) deduplicates on read (`GROUP BY/FINAL`), always per-board, never a global scan.

6.2 Medallion layout

`BRONZE = scrape_raw` (immutable) → `SILVER = jobs_parsed` (batch, later, replayable from Bronze at zero re-scrape cost) → `GOLD` (product-facing). **Boundary law:** scrape phase = every-

thing that needs the *network*; batch phase = everything that needs only the *bytes*. Detail-fetching needs the network → scraper. Field mapping needs only bytes → batch.

6.3 Temporal metadata via CDC

Requirement: all Temporal execution metadata queryable in ClickHouse next to the scrape evidence. Pipeline: **Debezium (Kafka Connect) on Temporal's Postgres → Kafka → ClickHouse**.

The one crucial aiming instruction: Temporal's core history tables store serialized protobuf blobs — CDC on those yields unreadable garbage. The *visibility* tables (`executions_visibility`) are plain rows: workflow id, type, status, start/close times, failure summary. **CDC the visibility tables.** Result: every workflow lifecycle event lands in `temporal_meta`, joinable to `scrape_evidence` by run and board.

7 Part 3 — Temporal, Fail-Loud

Temporal's whole job: **schedule, fan out, run, record.** It is built to fail visibly; when it breaks, we step in, figure it out, and go. No defensive tuning.

7.1 Topology

```
Schedule (per platform, staggered)
  -> PlatformRun parent workflow      (one per platform, never a mega-parent)
       activity plan_run  -> frozen RunPlan (board list + seen-sets + sizes)
       -> children: cheap families batched N boards/child (N is a sized
          hypothesis); heavy families (paged+detail, browser) 1 board/child
```

Parent-per-platform keeps each history far below Temporal's ∼50K-event cap and matches the real failure domain (platforms break platform-wide). The parent does not wait on stragglers; a run finishes with its failures recorded, not resolved.

7.2 The six things to know — reclassified

Two are laws, one is off by doctrine, three are ratios owned by the calibrator (§8.2):

#	Item	Status
1	Workflow vs Activity	**LAW.** Workflows decide (deterministic); activities do (network, disk). No HTTP, no clocks, no randomness in workflow code.
2	Worker slots	**HYPOTHESIS — never a guard.** Start from a stated guess. An OOM-kill is a *loud, recorded event*, not a disaster to prevent upfront: k8s restarts the pod, the interrupted boards surface as ghosts (**G2**), and the calibrator resizes from the data (§8.2). Nothing is defensively capped.
3	Heartbeat	**OFF by doctrine.** No dead-worker detector in Temporal. The daily absence gate (**G2**) detects ghosts at daily latency — accepted deliberately.
4	`start_to_close`	**Absurd by design** (30 days — the SDK requires *some* value; ours exists but never fires). Duration drift is reported by the calibrator, not enforced by a leash.
5	Retries	**MANUAL.** `maximum_attempts=1` explicitly — Temporal's *default is infinite retries*, so "no policy" must be written down as one attempt. Failure lands as data; Airflow re-runs on its cadence.
6	Payloads	**LAW.** Activities write raw data to ClickHouse themselves and return only counts. Data never travels through Temporal — Temporal carries decisions and numbers. If the law is broken, Temporal breaks loudly (2MB hard cap) and the audit gate (**G7**) names the culprit.

7.3 Determinism and the RunPlan

Workflows never read live config (determinism). The first activity `plan_run` snapshots policy + ledger into a frozen **RunPlan**, which the parent executes and which is written to ClickHouse. Every run records exactly the plan it ran under: a weird Thursday is `diff(RunPlan_thu, RunPlan_wed)`, not archaeology.

8 Part 4 — Airflow: Janitor, Calibrator, Auditor

Airflow (Apache Foundation — consistent with the standing foundation-bias tool rule) runs on KubernetesExecutor: each task is a pod that lives and dies; idle Airflow is just scheduler + webserver + its metadata Postgres. It has three roles and owns all judgment in the system.

8.1 Janitor (daily)

Reads `scrape_evidence` + `temporal_meta`: which boards failed, which are absent, which gates fired. Re-runs failures by starting fresh Temporal workflows (never by resurrecting old ones). Runs the Bronze → Silver parse via `KubernetesPodOperator`.

8.2 Calibrator (weekly, or on demand)

Closes the loop on *no invented constants*. Every sized quantity is a hypothesis row (value, rationale, date); calibration DAGs recompute recommendations from data:

Quantity	Signal	Computation
Worker slots per queue	p95 memory per scrape per family; observed OOM-kill events	recommend the next hypothesis from data (OOMs included as signal, not treated as failure to prevent)
Batch size per platform	EWMA board duration	boards per child s.t. child duration $\approx$ target
Nightly budget spend	requests + seconds per platform per night	report spend vs. intent; flag platforms starving the floor rule
Re-run cadence value	success-rate-on-rerun	90% pass on rerun $\Rightarrow$ blips, daily is right; 20% $\Rightarrow$ those boards are dying differently
Duration drift	p50/p99 per family per platform	report only — no leash to tighten

Recommendations become updated policy rows (read by the next `plan_run`) or, when a change implies k8s resources, a *proposed gitops MR* — Airflow never mutates the cluster directly. Every derived value carries hardcoded *clamps* (min/max bounds): bounds are physics, values are learned; one weird night must not teach the system garbage.

8.3 Scheduling brain (computed nightly, executed by Temporal)

The classic crawler-revisit problem, solved the boring way:

- Per board, EWMAs from the ledger: `cost_b` (requests + seconds), `yield_b` (*new* jobs), `score_b = yield_b / cost_b`.
- **Greedy knapsack per platform:** sort boards by score, scrape until the platform's nightly budget (a human *intent*, set in Django) is spent.
- **Floor rule (non-negotiable):** every board scraped at least once every N days regardless of score — a dead board can wake up, and without the floor the model never learns it did. N is itself calibrated from observed wake-up rates per platform.

Emergent, zero per-board config: hot boards drift to nightly, dead boards decay to weekly, newly-churning boards are promoted within ~3 runs. Chronic failers deprioritize *themselves*: their cost rises, their yield is zero, their score collapses — economics is the retry policy across nights; the floor guarantees nobody is forgotten forever.

8.4 Incremental extraction (the dedup mechanism)

1. **Always scrape lists blind** — cheap; keeps the index complete.
2. **Fetch details only for unseen stubs.** `plan_run` queries the board's known `stub_digest` set (per-board, dedup-on-read) and passes it *into* the activity. The scraper stays a pure function `(board, known_digests, ctx)` $\rightarrow$ `RawResult`; ClickHouse owns state.
3. **Periodic full re-fetch** per board heals the drift incremental extraction cannot see (a job edited without its stub changing). The interval is calibrated from measured edit rates: on every full re-fetch, count how many "unchanged" stubs had changed details — the refetch generates its own tuning signal.

Effect: ~95% compute cut on steady boards, compounding with §8.3. Accepted loss: silently-edited details are stale up to the refetch interval; job ads are post/expire dominated.

8.5 Cold start — eliminated structurally

Boards arrive only via ownership flips, platform-by-platform (§12); the planner is seeded from atlas-kt history and known job counts; politeness caps live in the HTTP client and bind regardless

of planner state. The system is never simultaneously big and ignorant.

9 Part 5 — Django Control Plane

Boards inventory (the growing table), per-platform switches (kill switch: pause a platform NOW from the admin), **owner** per board (§12), and the human *intents*: nightly budget per platform, schedule times. Everything else that was once "config" is either a law (in code), a doctrine (written here), or a calibrated hypothesis (computed by Airflow). The admin edits intents; the data audits them.

10 Observability = Validation Gates

Doctrine: *a signal that does not change a decision does not exist.* No per-request metrics, no latency histograms of scraper internals, no log aggregation beyond what Temporal keeps, no new metrics stack: **the ledger is the telemetry**; Grafana reads ClickHouse; the Temporal UI shows live state.

Observability here is a chain of **gates** — explicit assertions with defined actions. Every gate evaluation writes a row to `gate_results`: the gates themselves are data, so "did the gates run" is itself checkable.

10.1 The gate chain

Gate	Assertion	On failure	Where
G1 Delivery	child run: HTTP 200-class *and* `stubs_seen` > 0 on any board that had > 0 last run	workflow marked **FAILED**, visibly red — never a warning	Temporal child
G2 Presence	every board in last night's RunPlan has an evidence row today	board flagged *ghost*; re-run queued; this *is* the dead-worker detector (daily latency, by doctrine)	Airflow daily
G3 Volume	per platform: tonight's new-jobs count within historical band (3σ of trailing 30 runs) — *and* not ≈0 across all boards	**silent-success alarm** (the 2026-07-02 class, now watched, not just discouraged)	Airflow daily
G4 Landing	`scrape_raw` row count reconciles with `details_ok` claimed in evidence; digests unique	discrepancy alarm — evidence is lying or writes were lost	Airflow daily
G5 Parse	Bronze → Silver: parse-error rate within platform baseline; row counts reconcile; unknown-field ratio below drift threshold	parse halted for that platform; Bronze untouched; fix parser, replay	Airflow, per parse
G6 Ownership	no board has scrapes from both systems within 24h outside a declared parity window	loud page — this protects the *old* system's stability	Airflow daily
G7 Temporal audit	(via CDC) no workflow red without a matching evidence row; no history/payload size growing abnormally (= data leaking through Temporal, law #6 broken)	named-culprit alarm	Airflow weekly

10.2 Surfaces — capped

- **One dashboard** (one screen, no scrolling): one row per platform — boards planned / scraped / failed, new jobs, budget spent vs intent, gate summary. Green row = look away. Ten seconds over coffee.
- **One drill-down query** (parameterized, saved): failed/ghost boards for a platform with error class and evidence counts. Pull, never push.
- **One weekly trend view:** yield, cost per board, permanent-error rate, duration drift per platform over 30 days. Slow rot (Cloudflare tightening, an API decaying) shows here before it is an incident.

10.3 Alerts — four, total

Humans are interrupted only by: **(1)** run absent (platform's nightly run didn't start/finish); **(2)** G3 volume/silent-success; **(3)** G6 ownership violation; **(4)** G2 ghost-rate above platform baseline. Per-board failures are weather — the janitor handles them; nobody is paged. **Adding a fifth alert requires deleting one.** That rule is the whole discipline.

11 Deployment on Kubernetes

All changes via gitops MRs reconciled by ArgoCD — nothing is ever applied by hand. Internal GUIs (Airflow, Django admin) are tailnet-only behind the standard SSO path. Images are built on the Mac (pinned tags, tailnet registry — CI has no docker runner), `imagePullPolicy: IfNotPresent`. The architecture map is updated in the same unit of work as every deploy — a change without a map update is incomplete.

11.1 Reused (zero new stateful infrastructure)

1. **Temporal cluster** — already live; new namespace + two task queues (`scrape-http`, `scrape-browser`).
2. **ClickHouse** — the existing instance; new `scrape` database (§6).
3. **Kafka** — already present; carries the CDC stream.
4. **Keycloak + tailnet ingress** — standard auth for the two GUIs.

11.2 New components

1. **Scrape workers** — one Python image (scraper library + Temporal worker); two Deployments (http / browser). Replica counts, slots, and memory limits start as stated hypotheses and are re-sized from observed data — including OOM events — by the calibrator (§8.2); k8s restarts are part of the design, not incidents.
2. **Airflow** — official Apache chart, KubernetesExecutor, CNPG metadata Postgres.
3. **Debezium Connect** — on the existing Kafka, aimed at Temporal's *visibility* tables.
4. **Django** — one Deployment + CNPG Postgres.

11.3 Rollout order (one MR per step; map updated each step)

1. ClickHouse tables (minutes, zero risk).
2. Repo + CI (fixture gates) + worker image — scrapes nothing yet.
3. Worker Deployment + ONE platform schedule (greenhouse: simplest family).
4. Parity window vs. atlas-kt on a ≤25-board sample (§12).
5. Airflow + the daily janitor DAG + gates G2–G6.
6. Debezium CDC + gate G7 + the calibrator DAGs.
7. Flip greenhouse ownership; proceed platform-by-platform in board-count order.

12 Parallel Running — the Ownership Rule

Two systems hitting the same boards from the same egress IPs can trip rate limits and make the *old, working* system appear to degrade. Therefore, a rule, not advice:

> **THE OWNERSHIP RULE.** Every board row carries `owner ∈ {atlas-kt, new}`. A board is scraped by its owner and *only* its owner. No board is ever double-scraped — except inside a declared parity window.

- **Parity window** (per platform, during cutover only): ≤25 sample boards, both systems, runs ≥6h apart, new-system politeness cap halved, time-boxed (∼7 nights), then closed.

- **Cutover per platform:** port → fixture + parity gates green → flip `owner` for that platform's boards in one transaction. Rollback = flip back.
- **Enforcement is gate G6**, not good intentions.
- The old system needs zero code changes: its board list is already DB-driven; ownership filters at plan time.

Port discipline (mechanical, not aspirational): port order = board count descending (top ∼10 platforms cover ∼80% of boards); "ported" is a *defined state* (fixtures present + standalone run green + parity green N nights); one platform in flight at a time; **scheduler work beyond greedy+floor is forbidden until the top-10 platforms are live** — any earlier scheduler sophistication is procrastination wearing an algorithm costume.

13 Tradeoffs — accepted, eyes open

Choice	Cost accepted
Extract-don't-parse	No usable jobs until the parse batch exists; ∼2× storage (raw kept forever)
Incremental details	Silently-edited jobs stale up to the (calibrated) refetch interval
Fail-loud everywhere (no heartbeat, one attempt, no slot guards)	Detection latency up to a day; a wedged scrape holds a slot until noticed; pods may OOM-cycle while a sizing hypothesis is wrong — all accepted: these are loud, recorded events, and the daily pass plus calibration is the response
Daily janitor cadence	An unlucky board waits until the next pass — accounted for, never lost
Micro-batching cheap boards	Temporal UI shows batches, not boards; per-board truth lives only in ClickHouse — the ledger must be trusted
Greedy+floor scheduler	Provably suboptimal vs. bandits — deliberately; the delta does not pay for the complexity
Ownership rule	Parity evidence limited to small sample windows (full-fleet A/B was the interference risk)
One-platform-at-a-time port	Slower total port — half-ports are the failure mode being excluded
ClickHouse as single point of truth	Raw store + ledger + telemetry in one system: when ClickHouse is down, scraping state, planning, and dashboards degrade together

Residual risks (execution, not architecture): (1) the port grind dominates the calendar — the gates manage it, they don't shrink it; (2) discipline rules (no `if platform ==` in family files, four alerts max, the scheduler gate) are enforced by CI where possible and by the owner's discipline everywhere else — this is the single most likely path back to atlas-kt-shaped rot; (3) seeded priors can be wrong for platforms the old system handled badly — the floor rule bounds the damage to N days of staleness, and the first scrape corrects the model.

14 Open Decisions

1. Repo name and home (this system genuinely feeds the product, so the product group is legitimate — final call pending).
2. ClickHouse partitioning detail for `scrape_raw` (date + platform is the obvious start).
3. Parity thresholds: N nights, sample size, count-match tolerance defining "green."
4. Bronze → Silver parse design — explicitly deferred; nothing in Bronze constrains it.
5. Airflow vs. the in-flight Dagster port elsewhere in the stack: one janitor stack should win for both projects; decided once, not twice.

15 Summary

> Idempotent, incrementally-extracting ingestion jobs land immutable raw payloads to ClickHouse with content-hash keys and ingestion-time evidence; a seeded, self-calibrating planner schedules boards by yield-per-cost under human budget intents with a max-staleness floor; Temporal executes frozen per-run plans loudly and dumbly through family-templated, platform-thin scraper classes behind a hard board-ownership boundary; Airflow judges everything daily through explicit validation gates; and parsing is a replayable downstream batch concern.

Every sentence above was measured in the current codebase, paid for by a recorded incident, or is a standard pattern chosen over a clever one on purpose — and the v3 revisions (fail-loud Temporal, the calibrator, the gate chain) came from adversarial review of v1 and v2, on purpose too.